

Doc 7.2 — Object Static Methods & Object Copying

এই Doc-এ আমরা JavaScript Object-এর সবচেয়ে বেশি ব্যবহৃত কিছু built-in methods এবং object copying/merging-এর concepts শিখব।

এই Doc-এর মূল Topics

1. `Object.keys()`
2. `Object.values()`
3. `Object.entries()`
4. Object iteration
5. Object → Array conversion
6. `Object.assign()`
7. Object copying
8. Object merging
9. Object spread operator `...`
10. Spread vs `Object.assign()`
11. Shallow copy
12. Nested object reference problem
13. Immutable object update
14. React state object update patterns

1. কেন Object Methods দরকার?

ধরো:

```
const user = {  
  name: "Shariar",  
  age: 31,  
  role: "Developer"  
};
```

আমরা যদি চাই:

- শুধু keys
- শুধু values
- key-value pairs
- object loop করতে

- object copy করতে
- দুইটা object merge করতে

তাহলে JavaScript-এর built-in methods ব্যবহার করতে পারি।

সবচেয়ে গুরুত্বপূর্ণ:

```
Object.keys()  
Object.values()  
Object.entries()  
Object.assign()
```

2. `Object.keys()`

`Object.keys()` একটি object-এর **own enumerable property names** নিয়ে একটি array return করে।

Syntax:

```
Object.keys(object);
```

Example:

```
const user = {  
  name: "Shariar",  
  age: 31,  
  role: "Developer"  
};  
  
console.log(Object.keys(user));
```

Output:

```
["name", "age", "role"]
```

অর্থ্যাৎ:

```
Object  
  ↓  
Object.keys()  
  ↓  
Array of keys
```

3. `Object.keys()` Mental Model

ধরো:

```
const product = {  
  name: "Laptop",  
  price: 80000,  
  brand: "HP"  
};
```

তাহলে:

```
Object.keys(product);
```

শুধু property names দেবে:

```
["name", "price", "brand"]
```

Values দেবে না।

4. `Object.keys()` দিয়ে Object Loop

```
const user = {  
  name: "Shariar",  
  age: 31,  
  role: "Developer"  
};  
  
const keys = Object.keys(user);  
  
keys.forEach((key) => {  
  console.log(key);  
});
```

Output:

```
name
age
role
```

এটা useful যখন শুধু property names নিয়ে কাজ করতে হবে।

5. Object Key-এর Value Access করা

`Object.keys()` থেকে key পেয়ে bracket notation দিয়ে value পাওয়া যায়।

```
const user = {
  name: "Shariar",
  age: 31,
  role: "Developer"
};

Object.keys(user).forEach((key) => {
  console.log(key, user[key]);
});
```

Output:

```
name Shariar
age 31
role Developer
```

এখানে খুব important pattern:

```
Object.keys(object).forEach((key) => {
  console.log(object[key]);
});
```

6. `Object.values()`

`Object.values()` object-এর সব own enumerable property-এর **values** নিয়ে array return করে।

Syntax:

```
Object.values(object);
```

Example:

```
const user = {  
  name: "Shariar",  
  age: 31,  
  role: "Developer"  
};  
  
console.log(Object.values(user));
```

Output:

```
["Shariar", 31, "Developer"]
```

7. `Object.keys()` vs `Object.values()`

```
const user = {  
  name: "Shariar",  
  age: 31  
};
```

Keys

```
Object.keys(user);
```

Result:

```
["name", "age"]
```

Values

```
Object.values(user);
```

Result:

```
["Shariar", 31]
```

Mental model:

Object

```
├─ name → Shariar  
└─ age  → 31
```

keys()

↓

```
["name", "age"]
```

values()

↓

```
["Shariar", 31]
```

8. `Object.values()` দিয়ে Loop

```
const prices = {  
  laptop: 80000,  
  phone: 40000,  
  mouse: 2000  
};  
  
Object.values(prices).forEach((price) => {  
  console.log(price);  
});
```

Output:

```
80000  
40000  
2000
```

9. `Object.entries()`

`Object.entries()` object-এর key-value pair-গুলোকে nested array আকারে return করে।

Syntax:

```
Object.entries(object);
```

Example:

```
const user = {  
  name: "Shariar",  
  age: 31,  
  role: "Developer"  
};  
  
console.log(Object.entries(user));
```

Output:

```
[  
  ["name", "Shariar"],  
  ["age", 31],  
  ["role", "Developer"]  
]
```

10. `Object.entries()` Mental Model

Original object:

```
const user = {  
  name: "Shariar",  
  age: 31  
};
```

`Object.entries(user)`:

```
[
  ["name", "Shariar"],
  ["age", 31]
]
```

প্রতিটি pair:

```
[key, value]
```

11. `Object.entries()` দিয়ে Loop

```
const user = {
  name: "Shariar",
  age: 31,
  role: "Developer"
};

Object.entries(user).forEach(([key, value]) => {
  console.log(key, value);
});
```

Output:

```
name Shariar
age 31
role Developer
```

এখানে `[key, value]` হলো array destructuring।

Destructuring আমরা Object-এর পরের Doc-এ বিস্তারিত শিখব।

12. `Object.keys()`, `values()`, `entries()` একসাথে

ধরো:

```
const user = {  
  name: "Shariar",  
  age: 31  
};
```

`Object.keys()`

```
Object.keys(user);
```

```
["name", "age"]
```

`Object.values()`

```
Object.values(user);
```

```
["Shariar", 31]
```

`Object.entries()`

```
Object.entries(user);
```

```
[  
  ["name", "Shariar"],  
  ["age", 31]  
]
```

13. Quick Comparison

Method	Returns
<code>Object.keys(obj)</code>	keys-এর array
<code>Object.values(obj)</code>	values-এর array
<code>Object.entries(obj)</code>	<code>[key, value]</code> pairs-এর array

সব তিনটিই **array return** করে।

14. Object → Array Conversion

এই methods-এর একটি বড় সুবিধা হলো object-কে array-এর মতো process করা সহজ হয়।

ধরো:

```
const scores = {  
  math: 80,  
  english: 75,  
  science: 90  
};
```

Values array:

```
const scoresArray = Object.values(scores);
```

Result:

```
[80, 75, 90]
```

এখন array-এর methods ব্যবহার করা যাবে:

```
const total = Object.values(scores).reduce((sum, score) => {  
  return sum + score;  
}, 0);  
  
console.log(total);
```

Output:

```
245
```

15. Object-এ `map()` Directly কেন নেই?

এটা কাজ করবে না:

```
user.map(...)
```

কারণ normal object array নয়।

Object-কে আগে array representation-এ নিতে হয়।

যেমন:

```
Object.keys(user)
```

অথবা:

```
Object.values(user)
```

অথবা:

```
Object.entries(user)
```

তারপর array methods ব্যবহার করা যায়।

16. Object.entries() + Array Methods

এটা খুব powerful pattern:

```
const prices = {
  laptop: 80000,
  phone: 40000,
  mouse: 2000
};

const expensiveItems = Object.entries(prices)
  .filter(([name, price]) => price > 10000);

console.log(expensiveItems);
```

Result:

```
[
  ["laptop", 80000],
  ["phone", 40000]
]
```

17. Object.entries() থেকে আবার Object

`Object.entries()` object-কে array বানায়।

আবার `Object.fromEntries()` array of pairs-কে object বানাতে পারে।

```
const entries = [
  ["name", "Shariar"],
  ["age", 31]
];

const user = Object.fromEntries(entries);

console.log(user);
```

Result:

```
{
  name: "Shariar",
  age: 31
}
```

Mental model:

```
Object
  ↓
Object.entries()
  ↓
Array of [key, value]
  ↓
Object.fromEntries()
  ↓
Object
```

এটি object transformation-এর জন্য খুব useful।

18. `Object.assign()`

`Object.assign()` একটি বা একাধিক source object-এর properties একটি target object-এ copy করে।

Syntax:

```
Object.assign(target, source1, source2);
```

Example:

```
const target = {};  
  
const source = {  
  name: "Shariar",  
  age: 31  
};  
  
Object.assign(target, source);  
  
console.log(target);
```

Result:

```
{  
  name: "Shariar",  
  age: 31  
}
```

19. `Object.assign()` Copy করার জন্য

```
const user = {  
  name: "Shariar",  
  age: 31  
};  
  
const copy = Object.assign({}, user);
```

এখন:

```
console.log(copy);
```

Result:

```
{
  name: "Shariar",
  age: 31
}
```

20. `Object.assign()` কি Deep Copy?

না।

`Object.assign()` shallow copy করে।

এটা খুব important।

```
const user = {
  name: "Shariar",
  address: {
    city: "Dhaka"
  }
};

const copy = Object.assign({}, user);
```

Outer object নতুন হলেও nested `address` object একই reference হতে পারে।

21. `Object.assign()` দিয়ে Merge

একাধিক object merge করা যায়।

```
const user = {
  name: "Shariar"
};

const details = {
  age: 31,
  role: "Developer"
};

const result = Object.assign({}, user, details);

console.log(result);
```

Result:

```
{
  name: "Shariar",
  age: 31,
  role: "Developer"
}
```

22. Same Property হলে কী হয়?

```
const user = {
  name: "Shariar",
  age: 31
};

const update = {
  age: 32
};

const result = Object.assign({}, user, update);
```

Result:

```
{
  name: "Shariar",
  age: 32
}
```

পরের source-এর property আগের property overwrite করে।

23. `Object.assign()` Order Matters

```
const result = Object.assign(
  {},
  { age: 30 },
  { age: 31 },
  { age: 32 }
);
```

Final:

```
{
  age: 32
}
```

কারণ শেষ source-এর value final value হয়।

24. Object Spread Operator

Modern JavaScript-এ object copy এবং merge করার সবচেয়ে common pattern হলো **object spread**।

Syntax:

```
const copy = {
  ...original
};
```

Example:

```
const user = {  
  name: "Shariar",  
  age: 31  
};  
  
const copy = {  
  ...user  
};
```

Result:

```
{  
  name: "Shariar",  
  age: 31  
}
```

25. Spread দিয়ে Object Copy

```
const user = {  
  name: "Shariar",  
  age: 31  
};  
  
const copy = { ...user };  
  
console.log(copy === user);
```

Output:

```
false
```

কারণ নতুন outer object তৈরি হয়েছে।

26. Spread দিয়ে Object Merge

```
const user = {
  name: "Shariar"
};

const details = {
  age: 31,
  role: "Developer"
};

const result = {
  ...user,
  ...details
};
```

Result:

```
{
  name: "Shariar",
  age: 31,
  role: "Developer"
}
```

27. Spread Merge-এর Order

```
const result = {
  age: 30,
  ...user
};
```

যদি `user` -এর মধ্যে `age: 31` থাকে, তাহলে:

```
age: 31
```

হবে।

কারণ spread পরে এসেছে।

28. Reverse করলে

```
const result = {  
  ...user,  
  age: 30  
};
```

এখন final:

```
age: 30
```

কারণ explicit `age: 30` পরে এসেছে।

Golden Rule

*Object spread-এ যে **property** পরে আসে, সেটাই **final value** দেয়।*

29. Object Spread দিয়ে Property Update

React-এ সবচেয়ে important pattern:

```
const user = {  
  name: "Shariar",  
  age: 31  
};  
  
const updatedUser = {  
  ...user,  
  age: 32  
};
```

Result:

```
{  
  name: "Shariar",  
  age: 32  
}
```

পুরনো object mutate করা হয়নি।

30. Property Add করা

```
const user = {
  name: "Shariar",
  age: 31
};

const updatedUser = {
  ...user,
  role: "Developer"
};
```

Result:

```
{
  name: "Shariar",
  age: 31,
  role: "Developer"
}
```

31. Property Remove করার Immutable Pattern

Spread দিয়ে property সরাসরি remove করা যায় না।

সাধারণত destructuring + rest pattern ব্যবহার করা হয়।

```
const user = {
  name: "Shariar",
  age: 31,
  role: "Developer"
};

const { role, ...userWithoutRole } = user;

console.log(userWithoutRole);
```

Result:

```
{
  name: "Shariar",
  age: 31
}
```

Object destructuring/rest pattern পরের Doc-এ বিস্তারিত আসবে।

32. Spread বনাম `Object.assign()`

দুটো দিয়েই shallow copy এবং merge করা যায়।

Spread

```
const copy = {
  ...user
};
```

`Object.assign()`

```
const copy = Object.assign({}, user);
```

দুটোর সাধারণ result একই ধরনের।

33. Comparison Table

Feature	Spread	<code>Object.assign()</code>
Copy object	✓	✓
Merge objects	✓	✓
Shallow copy	✓	✓
Nested deep copy	✗	✗
Readability	খুব ভালো	ভালো
Modern code	খুব common	Common
Target object mutate	✗	Target object mutate করে

শেষের point খুব important।

34. `Object.assign()` Target Mutate করে

দেখো:

```
const target = {  
  name: "Old"  
};  
  
const source = {  
  name: "New"  
};  
  
Object.assign(target, source);  
  
console.log(target);
```

Result:

```
{  
  name: "New"  
}
```

Original `target` object-ই modify হয়েছে।

35. Spread নতুন Object তৈরি করে

```
const target = {  
  name: "Old"  
};  
  
const source = {  
  name: "New"  
};  
  
const result = {  
  ...target,  
  ...source  
};
```

এখানে `target` mutate হয়নি।

```
console.log(target.name);
```

Output:

```
Old
```

আর:

```
console.log(result.name);
```

Output:

```
New
```

36. Shallow Copy কী?

Shallow copy মানে:

শুধু **top-level properties** নতুন object-এ copy করা হবে।

Nested object থাকলে nested object-এর reference একই থাকতে পারে।

Example:

```
const user = {  
  name: "Shariar",  
  address: {  
    city: "Dhaka"  
  }  
};  
  
const copy = {  
  ...user  
};
```

এখানে:

```
user  
├─ name → "Shariar"  
└─ address ───┐  
                ↓  
              { city: "Dhaka" }  
  
copy  
├─ name → "Shariar"  
└─ address ───┐  
                ↑  
              same reference
```

37. Shallow Copy Demonstration

```
const user = {
  name: "Shariar",
  address: {
    city: "Dhaka"
  }
};

const copy = {
  ...user
};

copy.name = "Duke";

console.log(user.name);
```

Output:

```
Shariar
```

কারণ `name` primitive value এবং outer property নতুন object-এ copy হয়েছে।

38. Nested Reference Problem

এখন:

```
copy.address.city = "Rangpur";
```

তারপর:

```
console.log(user.address.city);
```

Output:

```
Rangpur
```

কেন?

কারণ:

```
user.address === copy.address
```

ফলাফল:

```
true
```

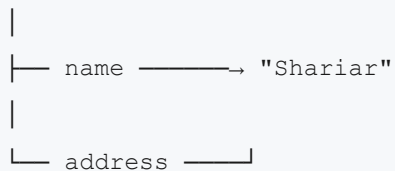
দুটো object একই nested `address` reference ধরে আছে।

39. Shallow Copy Visual

Original user



Copied user



Outer object আলাদা।

Nested object একই।

40. Nested Object Immutable Update

Nested property update করতে nested level-ও copy করতে হবে।

```
const user = {
  name: "Shariar",
  address: {
    city: "Dhaka",
    country: "Bangladesh"
  }
};

const updatedUser = {
  ...user,
  address: {
    ...user.address,
    city: "Rangpur"
  }
};
```

এখন:

```
console.log(user.address.city);
```

Output:

Dhaka

আর:

```
console.log(updatedUser.address.city);
```

Output:

Rangpur

41. কেন দুইবার Spread?

কারণ দুইটা level আছে:

```
user
└─ address
    └─ city
```

তাই:

```
const updatedUser = {
  ...user,
  address: {
    ...user.address,
    city: "Rangpur"
  }
};
```

প্রথম spread:

```
...user
```

Outer object copy করে।

দ্বিতীয় spread:

```
...user.address
```

Nested address object copy করে।

42. React-এ Object State

React-এ state object খুব common:

```
const [user, setUser] = useState({
  name: "Shariar",
  age: 31
});
```

ধরো age update করতে হবে।

ভুল approach:

```
user.age = 32;
```

এভাবে directly state mutate করা উচিত নয়।

43. React-এ Correct Object Update

```
setUser({
  ...user,
  age: 32
});
```

এখানে:

1. পুরনো user-এর properties copy হচ্ছে
2. `age` overwrite হচ্ছে
3. নতুন object তৈরি হচ্ছে
4. React নতুন reference পাচ্ছে

44. React Nested State Update

```
const [user, setUser] = useState({
  name: "Shariar",
  address: {
    city: "Dhaka"
  }
});
```

City update:

```
setUser({
  ...user,
  address: {
    ...user.address,
    city: "Rangpur"
  }
});
```

এটাই immutable nested update pattern।

45. Functional State Update

যদি নতুন state পুরনো state-এর উপর নির্ভর করে, functional updater আরও safe pattern:

```
setUser((previousUser) => ({
  ...previousUser,
  age: previousUser.age + 1
}));
```

এখানে:

```
previousUser
```

হলো previous state।

46. Object Copy কখন ব্যবহার করব?

Object copy useful যখন:

- state update করতে হবে
 - original object preserve করতে হবে
 - API data transform করতে হবে
 - নতুন version তৈরি করতে হবে
 - configuration extend করতে হবে
 - object merge করতে হবে
 - function-এর ভিতরে safe update করতে হবে
-

47. Real-World Example — API Data Update

```
const user = {  
  id: 1,  
  name: "Shariar",  
  role: "Developer",  
  active: true  
};  
  
const updatedUser = {  
  ...user,  
  active: false  
};
```

Original:

```
user.active
```

Result:

```
true
```

Updated:

```
updatedUser.active
```

Result:

```
false
```

48. Real-World Example — Default Configuration

```
const defaultConfig = {
  theme: "light",
  language: "en",
  notifications: true
};

const userConfig = {
  theme: "dark"
};

const finalConfig = {
  ...defaultConfig,
  ...userConfig
};
```

Result:

```
{
  theme: "dark",
  language: "en",
  notifications: true
}
```

এখানে user-এর custom config default config-কে override করেছে।

49. Object.keys() দিয়ে Validation

```
const user = {
  name: "Shariar",
  email: "shariar@example.com"
};

const requiredFields = ["name", "email"];

const allPresent = requiredFields.every((field) => {
  return Object.hasOwn(user, field);
});

console.log(allPresent);
```

Output:

```
true
```

এ ধরনের pattern form validation বা API validation-এ useful।

50. Object.values() দিয়ে Condition Check

```
const permissions = {
  read: true,
  write: true,
  delete: false
};

const hasPermission = Object.values(permissions).some(
  (permission) => permission === true
);

console.log(hasPermission);
```

Output:

```
true
```

51. Object.entries() দিয়ে Dynamic Rendering

React-এর context-এ:

```
const user = {  
  name: "Shariar",  
  age: 31,  
  role: "Developer"  
};
```

Render করার জন্য:

```
Object.entries(user).map(([key, value]) => (  
  <p key={key}>  
    {key}: {value}  
  </p>  
));
```

এখানে:

```
Object  
  ↓  
Object.entries()  
  ↓  
Array  
  ↓  
map()  
  ↓  
JSX
```

এই pattern React-এ খুব useful।

52. Object → Array → Object Transformation

এটা advanced কিন্তু খুব useful pattern।

```
const user = {  
  name: "Shariar",  
  age: 31,  
  role: "Developer"  
};
```

প্রথমে entries:

```
const entries = Object.entries(user);
```

তারপর transform:

```
const updatedEntries = entries.map(([key, value]) => {  
  if (key === "age") {  
    return [key, value + 1];  
  }  
  
  return [key, value];  
});
```

আবার object:

```
const updatedUser = Object.fromEntries(updatedEntries);
```

Result:

```
{  
  name: "Shariar",  
  age: 32,  
  role: "Developer"  
}
```

Mental model:

```
Object
  ↓
  entries()
  ↓
  Array
  ↓
  map/filter
  ↓
  fromEntries()
  ↓
  Object
```

53. `Object.keys()` -এর একটি Important Point

`Object.keys()` শুধু **own enumerable properties** return করে।

সাধারণ object-এর ক্ষেত্রে সহজভাবে মনে রাখো:

Object-এর নিজের enumerable keys-এর array দেয়।

Prototype-এর inherited properties সাধারণত এতে আসে না।

54. Empty Object

```
const user = {};  
  
console.log(Object.keys(user));  
console.log(Object.values(user));  
console.log(Object.entries(user));
```

Output:

```
[]  
[]  
[]
```

তিনটিই empty array return করবে।

55. Object Methods কি Original Object Mutate করে?

সাধারণভাবে:

```
Object.keys()  
Object.values()  
Object.entries()
```

original object modify করে না।

এগুলো data-এর array representation return করে।

অন্যদিকে:

```
Object.assign(target, source);
```

target object modify করতে পারে।

এই difference মনে রাখো।

56. Spread এবং Nested Object

এই code:

```
const copy = {  
  ...user  
};
```

মানে:

"পুরো object-এর সব nested structure recursively clone করে"

এটা নয়।

এটা শুধু shallow copy।

Deeply nested data থাকলে nested reference issue থাকবে।

57. Deep Copy সম্পর্কে Basic Idea

Deep copy মানে nested objects/arrays-সহ নতুন references তৈরি করা।

Modern JavaScript-এ একটি option:

```
const copy = structuredClone(original);
```

Example:

```
const user = {
  name: "Shariar",
  address: {
    city: "Dhaka"
  }
};

const copy = structuredClone(user);

copy.address.city = "Rangpur";

console.log(user.address.city);
```

Output:

```
Dhaka
```

কারণ nested object-ও clone হয়েছে।

তবে `structuredClone()` সব ধরনের JavaScript value/function-এর জন্য universal solution নয়। Deep copy করার প্রয়োজন আছে কিনা আগে বুঝতে হবে।

58. Spread vs structuredClone ()

Feature	Spread	structuredClone ()
Outer object copy	✓	✓
Nested object copy	✗	✓
Shallow copy	✓	✗
Deep clone	✗	অনেক ক্ষেত্রে ✓
Lightweight update	✓	সাধারণত unnecessary
React state update	খুব common	সাধারণত দরকার হয় না

React state update-এর জন্য পুরো state deep clone করা সাধারণত প্রয়োজন হয় না।
যে nested level update করবে, সেই level copy করাই সাধারণ immutable pattern।

59. Common Mistake — Spread মানেই Deep Copy

ভুল:

```
const copy = {  
  ...user  
};
```

তারপর ধরে নেওয়া:

```
copy.address !== user.address
```

সবসময় সত্য নয়।

Nested object হলে:

```
copy.address === user.address
```

হতে পারে।

60. Common Mistake — Object.assign() দিয়ে Original Object Copy

এটা:

```
Object.assign(user, update);
```

original `user` modify করে।

যদি নতুন object চাও:

```
const updatedUser = Object.assign({}, user, update);
```

অথবা modern code-এ:

```
const updatedUser = {  
  ...user,  
  ...update  
};
```

61. Common Mistake — Spread Order ভুল

```
const updatedUser = {  
  age: 32,  
  ...user  
};
```

যদি `user.age = 31` হয়, তাহলে final age হবে:

```
31
```

কারণ `...user` পরে এসেছে।

সঠিক update pattern:

```
const updatedUser = {  
  ...user,  
  age: 32  
};
```

62. Common Mistake — `Object.keys()` থেকে Value আশা করা

```
Object.keys(user);
```

এটা values দেয় না।

Values চাইলে:

```
Object.values(user);
```

Key-value pair চাইলে:

```
Object.entries(user);
```

63. Common Mistake — Object-এ Direct `map()`

ভুল:

```
user.map(...)
```

সাধারণ object-এর `map()` নেই।

সঠিক:

```
Object.entries(user).map(...)
```

অথবা প্রয়োজন অনুযায়ী:

```
Object.keys(user).map(...)
```

64. Quick Reference Table

Method / Syntax	কাজ
<code>Object.keys(obj)</code>	keys → array
<code>Object.values(obj)</code>	values → array
<code>Object.entries(obj)</code>	entries → array
<code>Object.fromEntries(arr)</code>	entries array → object
<code>Object.assign()</code>	copy/merge
<code>{ ...obj }</code>	shallow copy
<code>{ ...a, ...b }</code>	merge
<code>{ ...obj, key: value }</code>	immutable update
<code>structuredClone(obj)</code>	deep clone in supported cases

65. Practice Questions

Question 1

Output কী হবে?

```
const user = {
  name: "Shariar",
  age: 31,
  role: "Developer"
};

console.log(Object.keys(user));
```

Question 2

Output কী হবে?

```
const user = {  
  name: "Shariar",  
  age: 31  
};  
  
console.log(Object.values(user));
```

Question 3

Output কী হবে?

```
const user = {  
  name: "Shariar",  
  age: 31  
};  
  
console.log(Object.entries(user));
```

Question 4

Output কী হবে?

```
const user = {  
  name: "Shariar",  
  age: 31  
};  
  
const copy = {  
  ...user,  
  age: 32  
};  
  
console.log(user.age);  
console.log(copy.age);
```

Question 5

Output কী হবে?

```
const a = {
  name: "Shariar"
};

const b = {
  age: 31
};

const result = {
  ...a,
  ...b
};

console.log(result);
```

Question 6

Output কী হবে?

```
const user = {
  name: "Shariar",
  age: 31
};

const update = {
  age: 32
};

const result = {
  ...update,
  ...user
};

console.log(result.age);
```

Question 7

Output কী হবে?

```
const user = {  
  name: "Shariar",  
  address: {  
    city: "Dhaka"  
  }  
};  
  
const copy = {  
  ...user  
};  
  
copy.address.city = "Rangpur";  
  
console.log(user.address.city);
```

Question 8

Output কী হবে?

```
const target = {  
  name: "Old"  
};  
  
const source = {  
  name: "New"  
};  
  
const result = Object.assign(target, source);  
  
console.log(target.name);  
console.log(result.name);
```

66. Coding Challenges

Challenge 1 — Object Keys

এই object-এর সব keys print করো:

```
const product = {  
  name: "Laptop",  
  price: 80000,  
  brand: "HP",  
  stock: 5  
};
```

Expected:

```
name  
price  
brand  
stock
```

Challenge 2 — Object Values-এর Total

```
const prices = {  
  laptop: 80000,  
  phone: 40000,  
  mouse: 2000  
};
```

`Object.values()` ব্যবহার করে total price বের করো।

Expected:

```
122000
```

Challenge 3 — Object Merge

দুইটা object:

```
const user = {  
  name: "Shariar"  
};  
  
const details = {  
  age: 31,  
  role: "Developer"  
};
```

এগুলো merge করে নতুন object তৈরি করো।

Challenge 4 — Immutable Update

```
const user = {  
  name: "Shariar",  
  age: 31,  
  role: "Developer"  
};
```

নতুন object তৈরি করে age `32` করো।

Original object modify করা যাবে না।

Challenge 5 — Nested Immutable Update

```
const user = {  
  name: "Shariar",  
  address: {  
    city: "Dhaka",  
    country: "Bangladesh"  
  }  
};
```

নতুন object তৈরি করে শুধু city `"Rangpur"` করো।

Original `user` unchanged থাকতে হবে।

67. Interview Questions

1. `Object.keys()` কী return করে?

Object-এর own enumerable property keys-এর array।

2. `Object.values()` কী return করে?

Object-এর own enumerable property values-এর array।

3. `Object.entries()` কী return করে?

প্রতিটি property-এর `[key, value]` pair-এর array।

4. `Object.assign()` এবং spread-এর মধ্যে difference কী?

দুটোই shallow copy/merge করতে পারে।

তবে:

```
Object.assign(target, source);
```

target object modify করতে পারে।

আর:

```
const result = {  
  ...target,  
  ...source  
};
```

নতুন object তৈরি করে।

5. Spread কি deep copy করে?

না।

Spread **shallow copy** করে।

6. Nested object কেন একই reference থাকে?

কারণ shallow copy top-level properties copy করে; nested object-এর reference copy হয়।

7. React-এ spread operator কেন এত বেশি ব্যবহার হয়?

Immutable state update করার জন্য নতুন object তৈরি করা সহজ হয়।

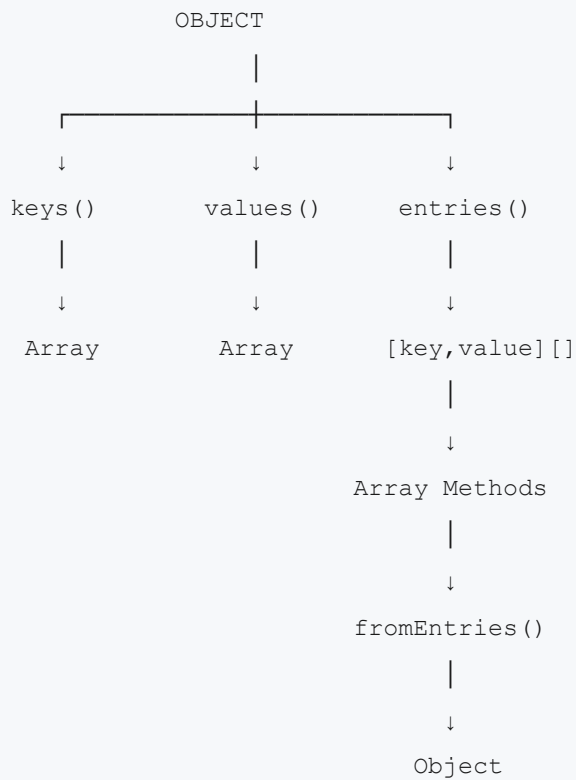
```
setUser({  
  ...user,  
  age: 32  
});
```

8. Object.entries() কেন useful?

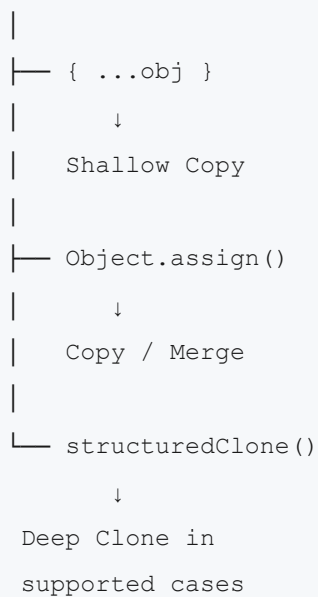
Object-কে `[key, value]` pair-এর array-তে convert করে, ফলে `map()`, `filter()`, `reduce()` ইত্যাদি array methods দিয়ে সহজে process করা যায়।

68. Mental Model

পুরো Doc-টা এই diagram দিয়ে মনে রাখতে পারো:



OBJECT COPYING



69. Golden Rules

🏆 Rule 1 `Object.keys()` → **keys**

🏆 Rule 2 `Object.values()` → **values**

🏆 Rule 3 `Object.entries()` → **key-value pairs**

🏆 Rule 4 Object-এ `map()` নেই; প্রয়োজন অনুযায়ী `Object.keys()`, `values()` বা `entries()` দিয়ে array বানিয়ে তারপর array methods ব্যবহার করো।

🏆 Rule 5 `{ ...obj }` → **shallow copy**

🏆 Rule 6 `{ ...a, ...b }` → **object merge**

🏆 Rule 7 Spread-এর পরে যে property আসে, সেটাই final value দেয়।

🏆 Rule 8 `Object.assign(target, source)` target object mutate করতে পারে।

🏆 Rule 9 Nested object থাকলে spread alone deep copy করে না।

🏆 Rule 10 React state update-এর সময় নতুন object তৈরি করতে spread খুব common pattern।

70. Final Mastery Checklist

এই Doc শেষ করার পর তোমার এগুলো confidently পারা উচিত:

- ☐ `Object.keys()` বুঝি
- ☐ `Object.values()` বুঝি
- ☐ `Object.entries()` বুঝি
- ☐ তিনটার difference বলতে পারি
- ☐ Object-কে array representation-এ convert করতে পারি
- ☐ `Object.fromEntries()` বুঝি
- ☐ Object entries-এর উপর `map/filter/reduce` চালাতে পারি
- ☐ `Object.assign()` বুঝি
- ☐ Object merge করতে পারি
- ☐ Object spread operator বুঝি
- ☐ Spread দিয়ে copy করতে পারি
- ☐ Spread দিয়ে merge করতে পারি
- ☐ Spread order-এর overwrite behavior বুঝি
- ☐ `Object.assign()` বনাম spread explain করতে পারি
- ☐ Shallow copy কী বুঝি
- ☐ Nested reference problem বুঝি
- ☐ Nested immutable update করতে পারি

- ☐ React state object update করতে পারি
 - ☐ `structuredClone()` সম্পর্কে basic ধারণা আছে
 - ☐ Object transformation pattern বুঝি
-



Next Doc



Doc 7.3 — Object Destructuring & Property Shortcuts

এখানে আমরা শিখব:

Object Destructuring

↓

```
const { name, age } = user
```

Renaming

↓

```
const { name: userName } = user
```

Defaults

↓

```
const { role = "user" } = user
```

Nested Destructuring

↓

```
const { address: { city } } = user
```

Function Parameter Destructuring

↓

```
function greet({ name }) {}
```

Rest Pattern

↓

```
const { name, ...others } = user
```

Shorthand Property Names

↓

```
const name = "Shariar";  
const user = { name };
```

Shorthand Methods

↓

```
const user = {  
  greet() {}  
};
```

এই Doc-টা শেষ করলে Object syntax অনেক বেশি clean এবং professional লাগবে।